

INF-2700 Mandatory Assignment No. 2

DBMS Elements

Deadline: Friday, 2 October 2026, 23:59

INF-2700 Mandatory Assignment No. 2

Deadline: Friday, 2 October 2026, 23:59

Submission: Canvas

DBMS Elements

In this assignment, you will implement in C some basic elements of a database management system (DBMS).

By working on the tasks, you can gain first-hand experience with how DBMS elements work. The implementation also provides practice with memory management and debugging. You are strongly encouraged to use debugging and program-analysis tools such as `gdb` and `valgrind`.

Code readability and documentation are important aspects of software development, and this assignment gives you an opportunity to practise both. You can also gain a better understanding of database performance.

Passing the Assignment

There are no specific requirements concerning which tasks you must complete or how much functionality you must implement.

To pass the assignment, you only need to submit something in Canvas before the deadline.

You are encouraged to work on as many of the tasks as you find useful. You may submit incomplete code, experiments, notes, a report, or other material showing your work.

You may ask for feedback on your work during the assignment period.

Getting Started

The base program and any supporting files are available through Canvas. Download the files and extract them into a suitable working directory.

The base program is located in the `db2700/` directory.

Spend some time studying the source code before making changes. You can then extend or modify the program while working on the tasks below. Aim to keep your program well structured and readable.

Task 1: Code Understanding

The first task is to understand how the base program works. The following activities may help:

- Read the generated documentation. From the `db2700/` directory, run:

```
make doc
```

This requires Doxygen to be installed. Open `doc/html/index.html` in a web browser to read the generated documentation for the API and data structures.

- Read the source code. Consider drawing diagrams while reading to clarify your understanding.
- Read the higher-level components to understand how they use the lower-level APIs. For example:
 - the front layer uses the schema layer;
 - the schema layer uses the pager layer.

- Run:

```
make
```

This produces three executable files:

```
– run_db2700
– run_testpager
– run_testschema
```

Run them in different ways to understand how the programs work. Use the `-h` option to display help:

```
./run_db2700 -h
./run_testpager -h
./run_testschema -h
```

- Make any modifications and experiments that help you understand the code.

- Keep backup copies of the original files or use any version-control system you prefer. Version control is optional and is not used for submission.
- Avoid including temporary table files, generated documentation, or executables in your final submission unless they are relevant to the work you want to show. These files can often be large and can normally be regenerated.
- Use the base program's `put_*_info()` procedures to observe intermediate system states.
- Attend the weekly interactive sessions and participate in discussions.
- Ask questions through the available course communication channels.
- Report any bugs you find in the base program, together with your debugging observations if possible.

Task 2: Extending the Types of Queries

In the base program, queries are restricted to equality searches on integer attributes. For a query of the form:

```
select attrs from table where attr op val;
```

the restrictions are:

- `attr` must be an integer attribute;
- `op` must be `=`;
- `val` must be an integer value.

Extend the base program so that queries on integer attributes are not restricted to equality searches. In addition to `=`, support the following operators:

- `<`
- `<=`
- `>`
- `>=`
- `!=`

Test your program using a relatively large table. To support testing, write code that can generate a table of an arbitrary size.

Task 3: Binary Search

The base program uses linear search to find records.

Extend the base program so that it can use binary search on an integer field. For this task, restrict the implementation to equality queries.

You may assume that:

- the records are ordered by the integer field being searched; and

- there are no duplicate values for the searched attribute.

To test your implementation, generate a table of an arbitrary size whose records are ordered by a chosen integer field and contain no duplicate values for that field.

Run queries on the ordered field that demonstrate both cases:

- the requested record exists;
- the requested record does not exist.

For a relatively large table, compare the performance of linear search and binary search. Do not measure performance using elapsed execution time. Instead, use the profiling capability provided by the base program.

Task 4: Comparison with a B⁺-Tree

This is a written task; no programming is required.

Consider an alternative in which the table is stored in a B⁺-tree-organised file.

Compare the following between your solution from Task 3 and the hypothetical B⁺-tree-organised file:

- the storage size of the table;
- the performance of queries.

State any assumptions you make, such as the fan-out of the B⁺-tree.

Feedback

You may ask for feedback while working on the assignment. Feedback can cover, for example:

- your understanding of the base program;
- your implementation approach;
- debugging problems;
- testing strategies;
- performance experiments;
- your Task 4 discussion;
- drafts or incomplete work.

Consult the course information in Canvas for the available feedback channels and response times.

Submission

Submit your work through **Canvas** before the deadline.

You may submit any material that represents your work, including incomplete work. Possible submission contents include:

- source code;
- test code or scripts;
- experimental results;
- notes;
- diagrams;
- a written report;
- a short text describing what you attempted.

If you include source code, it is helpful to include brief instructions explaining how to compile and run it.

You are encouraged, but not required, to submit a report named `report-assignment2.pdf`. A report could include:

- a description of your design and implementation;
- instructions for running your program and experiments;
- a comparison of linear-search and binary-search performance;
- special observations;
- lessons learned;
- problems encountered;
- your discussion for Task 4.

Generated executables and large database table files normally do not need to be submitted, provided they can be regenerated from your source code.

Remember: the only requirement for passing is to submit something in Canvas before the deadline.

Enjoy coding, and good luck!